UNITY GAME TEMPLATE – DIAMOND LOOP

USER GUIDE

CLAWBEAR GAMES
clawbeargames@gmail.com

Table of Contents

I. TEMPLATE INTRODUCTION 3

II. TEMPLATE SETUP 4

III. REQUIREMENT PACKAGES 4

IV. GAMEPLAY CUSTOMIZATION 5

1. INGAME MANAGER 5

V. LEVEL CUSTOMIZATION 6

VI. SERVICES CONFIGURATION 15

1. IAP MANAGER 15
2. AD MANAGER 17
3. UNITY AD CONTROLLER 19
4. ADMOB CONTROLLER 19
5. SOUND MANAGER 22
6. COIN MANAGER 23
7. SHARE MANAGER 23
8. LEADERBOARD MANAGER 24
9. NOTIFICATION MANAGER 25
10. DAILY REWARD MANAGER 28
11. CUSTOMIZING UI 28

I. TEMPLATE INTRODUCTION



💎 **Diamond Loop – Sort, Match & Clear the Board!**

Diamond Loop – Sort Puzzle is a complete block match & jam-style Unity puzzle template featuring a unique conveyor-based gameplay system combined with color sorting mechanics.

The board is formed by a 32x32 grid of 3D pixel cubes, creating colorful patterns that players must clear. At the bottom, cubes are collected into color-matching pots. Players tap a pot to send it onto a moving conveyor belt, where it travels across the board.

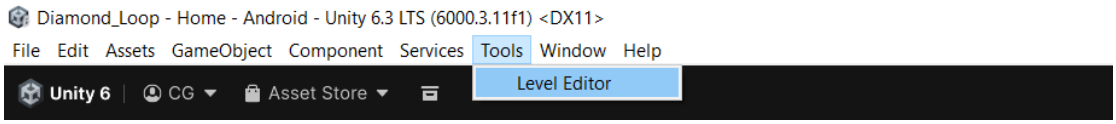
As the pot moves, matching color cubes jump from the grid into the pot, filling it up. Once full, the pot completes its cycle. The level is cleared when all cubes are collected and all pots are filled.

Strategic timing, positioning, and resource management are key, especially as special pot states and obstacles introduce new challenges.

With a fully built-in Level Editor, developers can easily configure cube layouts, conveyor behavior, pot states, and gameplay parameters to create hundreds of engaging puzzle levels.

Featuring clean lowpoly visuals, relaxing skybox environments, and Unity IAP v5 monetization support, Diamond Loop provides a powerful and scalable foundation for modern mobile puzzle games.

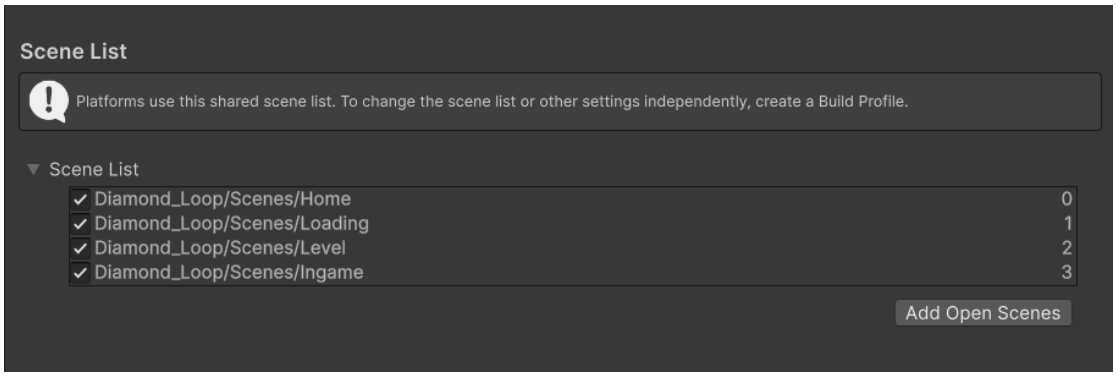
The Level Editor is a comprehensive Unity Editor tool designed for creating levels for games. Opening the Level Editor tool through Unity's menu: Tools > Level Editor



II. TEMPLATE SETUP

This template was designed for mobile (Android, iOS,…) so after imported the package to unity, you need to switch to Android or iOS.

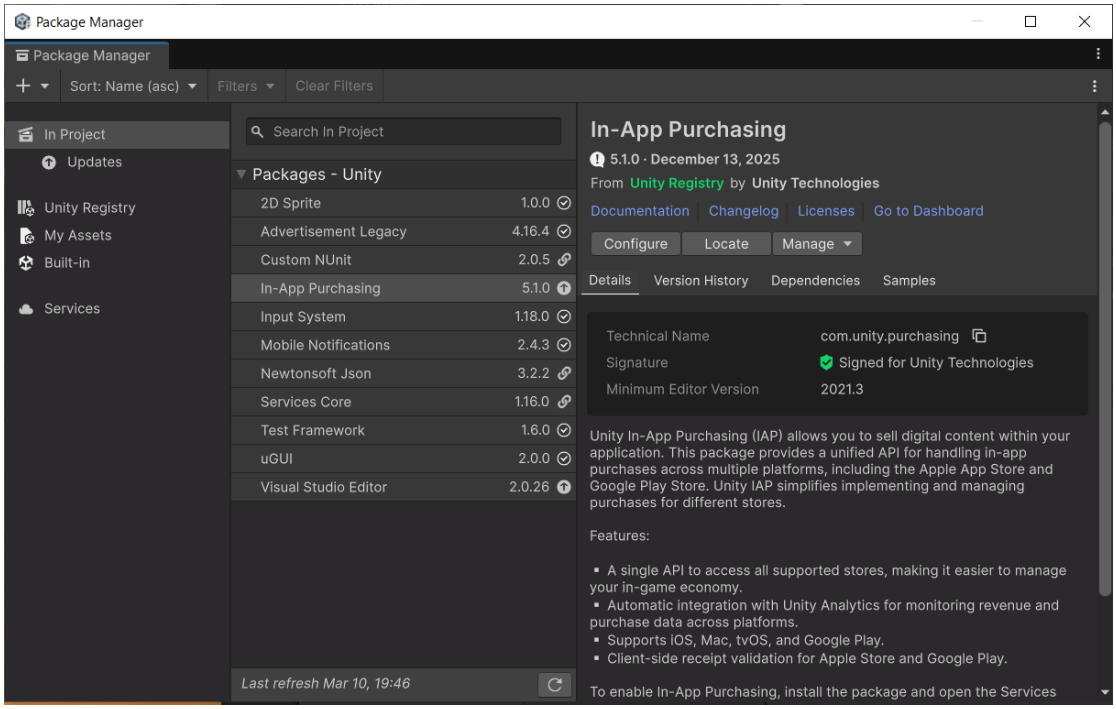
The template contains 4 scenes: Home, Loading, Level and Ingame under the path Assets/Diamond_Loop/Scenes. **You need to start from Home scene first. Do not start from other scenes.**



III. REQUIREMENT PACKAGES

When you open this template, at first you will some errors in Console window, that's because this template requires some packages to run. You have to install these packages bellow to have this template run smoothly. Please follow these instructions:

- Open Package Manager by go to Window/Package Manager. If you don't see Package Manager button, then you have to re-install your unity completely because it's the problem that Unity Technologies still not able to fix.

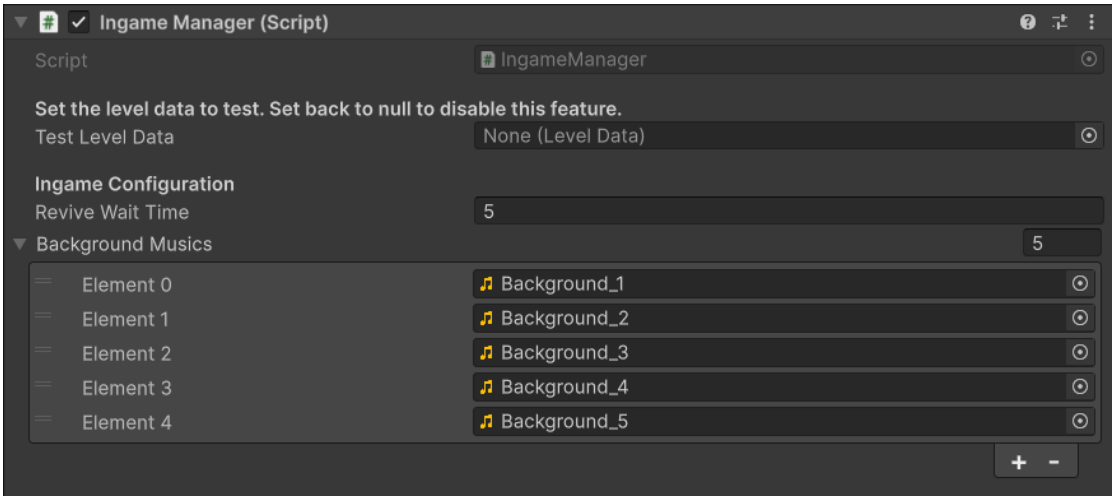


- These are packages you must need to install:
  - Advertisement version 4.16.4 or higher.
  - Mobile Notification version 2.4.3 or higher
  - In-App Purchasing version 5.1.0 or higher
  - 2D Sprite version 1.0.0 or higher
  - Unity UI 1.0.0 or higher

IV. GAMEPLAY CUSTOMIZATION

1. Ingame Manager

Most of important gameplay parameters can be configured within the IngameManager component which is attached to a game object also named IngameManager in the hierarchy. You can find IngameManager object in InGame scene.

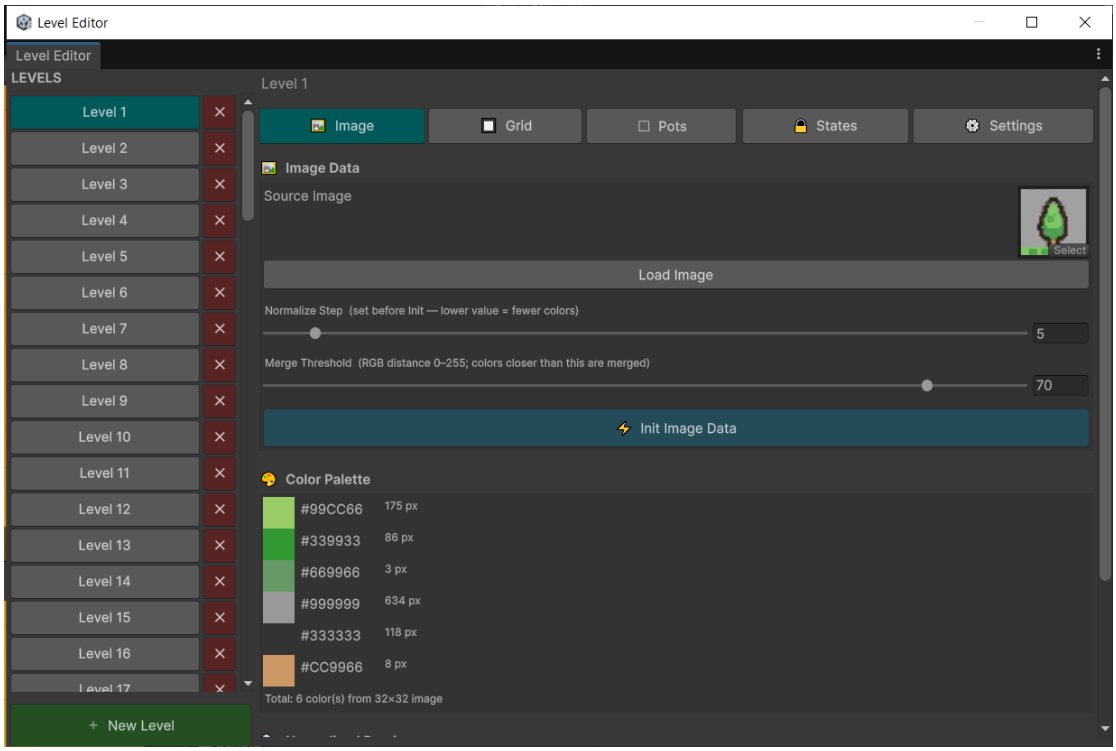


- ❖ *Test Level:* the level data you need to test before building the game. If you set it to null, the script will not load the testing level data.
- ❖ *Revive Wait Time:* the delay time for revive feature last.
- ❖ *Background Music:* the array of background music.

V. LEVEL CUSTOMIZATION

Overview

The Level Editor is a Unity Editor window (Tools > Level Editor) for creating and editing puzzle levels in Diamond Loop. Each level is stored as a LevelData ScriptableObject asset under Assets/Diamond_Loop/Resources/Levels/.



Levels are built in a fixed sequence across five modes:

Mode	Purpose

Image	Load a source image and extract a normalized color palette.
Grid	Set grid dimensions and toggle individual tiles on or off.
Pots	Configure per-color pot counts, choose a distribution method, and initialize pot placement.
States	Assign special states (Frozen, Locked, Decay) to individual pots.
Settings	Configure background colors, conveyor slot, power-ups, coin rewards, and star rating thresholds.

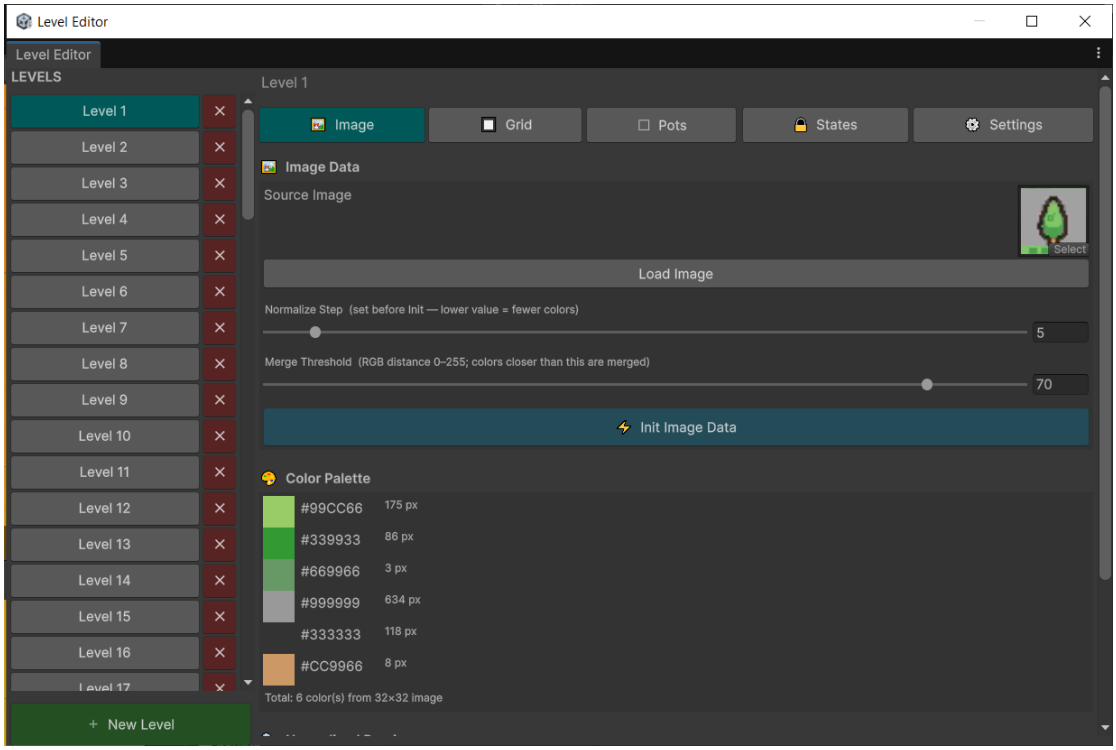
Recommended init order: Image → Grid → Pots → States. Skipping steps will show warning prompts.

Left Panel – Level List

The left panel (220 px wide) is a persistent level browser visible at all times regardless of the active mode.

Displays all LevelData assets sorted by level number. The currently selected level is highlighted in cyan. Each row contains two controls:

- **Level button** — Selects the level and loads it into the right panel for editing.
- ✕ **button** — Prompts for confirmation, then permanently deletes the asset from disk.



New Level button: Creates a new LevelData asset with the next available level number, a default 8 × 8 grid (all tiles enabled), and default power-up values. The new level is automatically selected for editing.

Right Panel – Level Params

The right panel contains the main editing area. When no level is selected it shows a placeholder prompt. Once a level is selected it displays:

- **Level title** — Shows the current level number.
- **Mode selector** — Five buttons for switching between modes. The active mode is highlighted in cyan.
- **Grid size info** — Shown in all modes except Image and Settings. In Grid Mode the sliders are interactive; in all other modes it is read-only.
- **Mode content** — The main editing area for the active mode, described below.

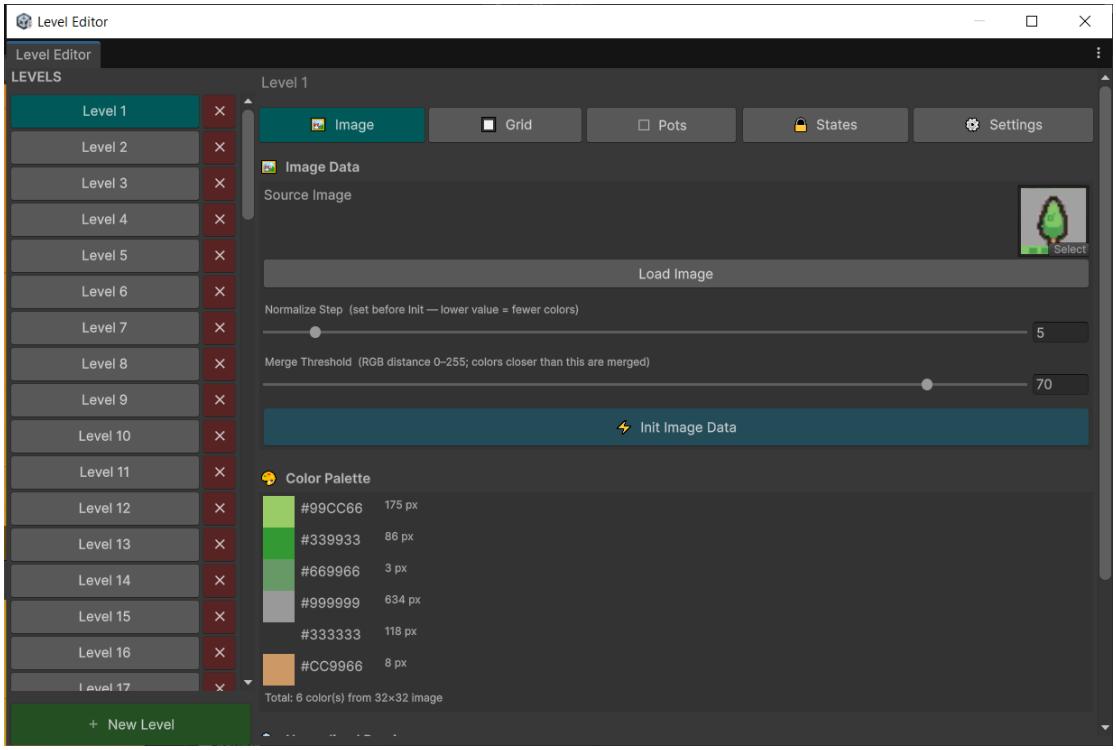
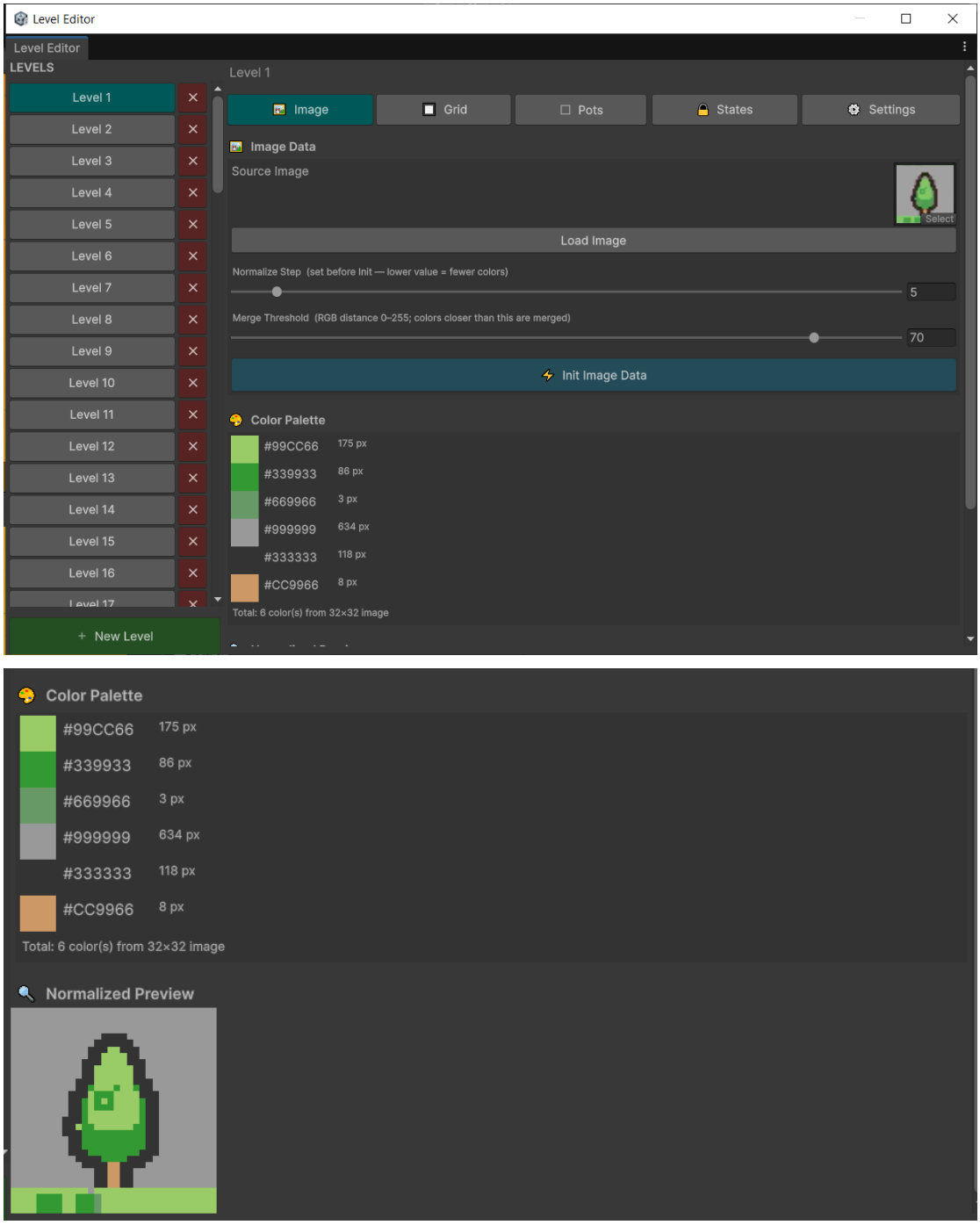


Image Mode

Used to load and process a reference image that drives color data for the level.

- **Source Image** — An object field and a Load Image button to pick a Texture2D asset from the project.
- **Normalize Step** — Slider (2–50). Controls color quantization resolution — lower values produce fewer distinct colors.
- **Merge Threshold** — Slider (0–80). RGB distance below which similar palette colors are merged into one.
- **Init Image Data** — Processes the source image and populates the color palette. Requires the texture to have Read/Write enabled in its import settings.

After initialization the panel shows a color palette swatch list (hex code + pixel count per color) and a small normalized image preview.

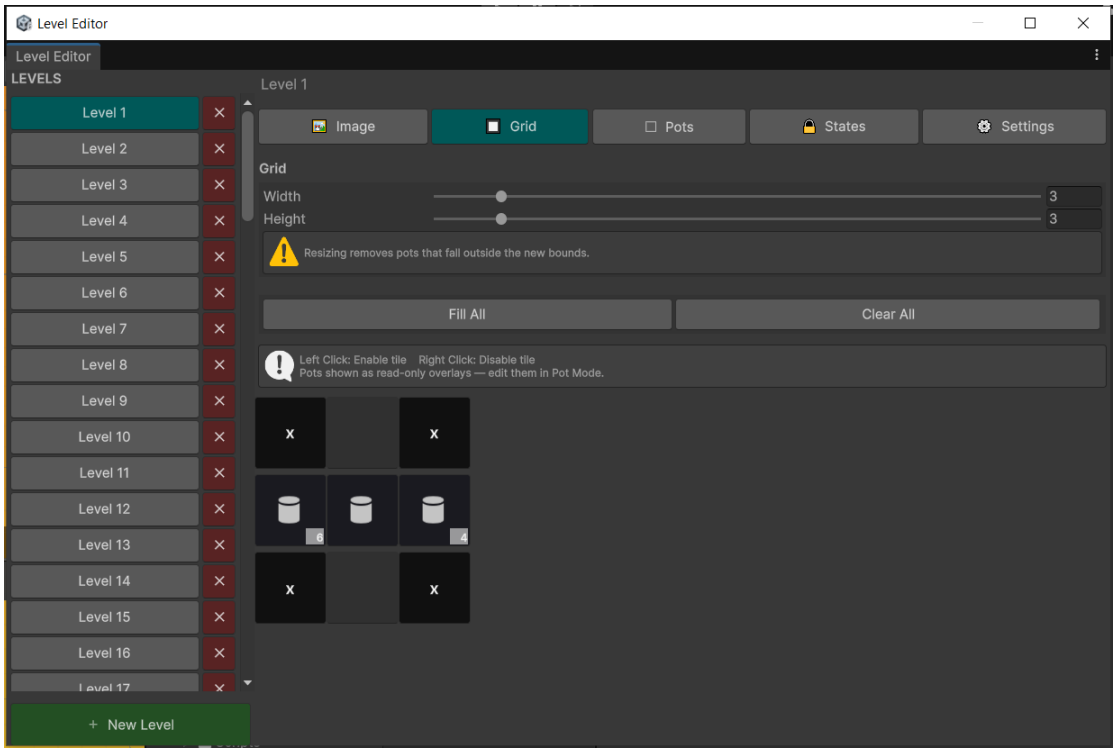


Grid Mode

Defines the playfield layout by toggling tiles on or off.

- **Width / Height sliders** — Resize the grid (min size defined by GridData.MIN_SIZE, max 20). Resizing preserves existing tile states where dimensions overlap and removes any pots that fall outside the new bounds.
- **Fill All** — Enables every tile in the grid.

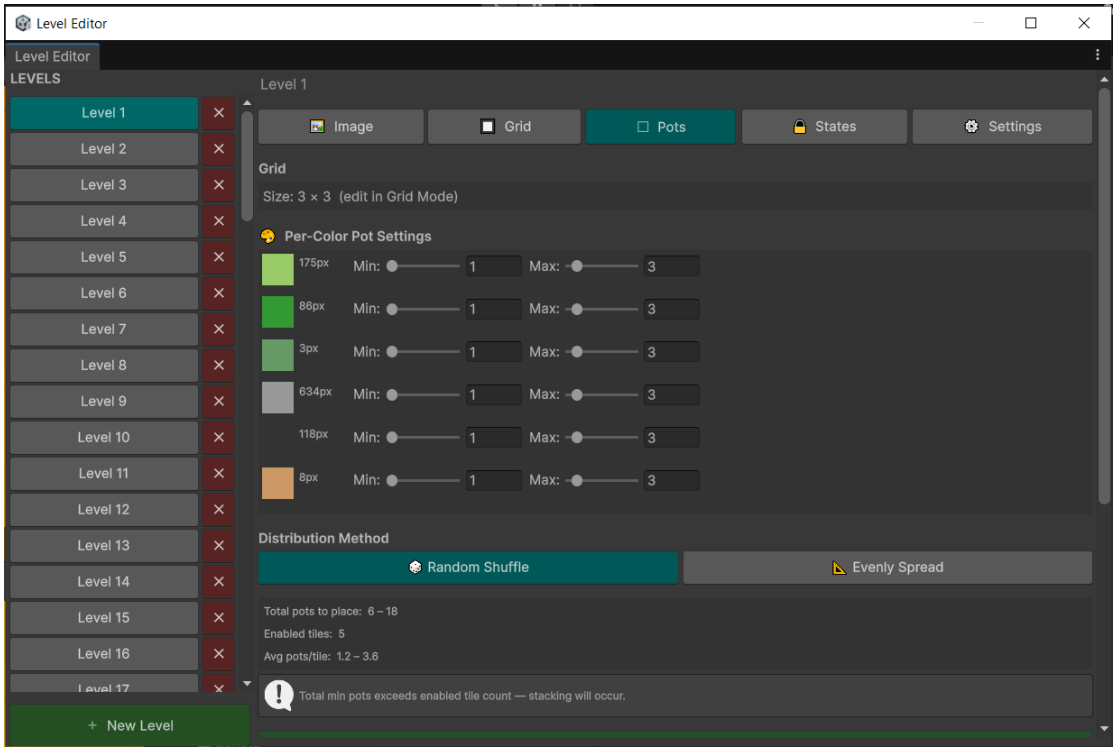
- **Clear All** — Disables all tiles and removes all pots after confirmation.
- **Grid interaction** — Left-click enables a tile; right-click disables it and removes any pots on it. Pots are shown as read-only cylinder overlays.

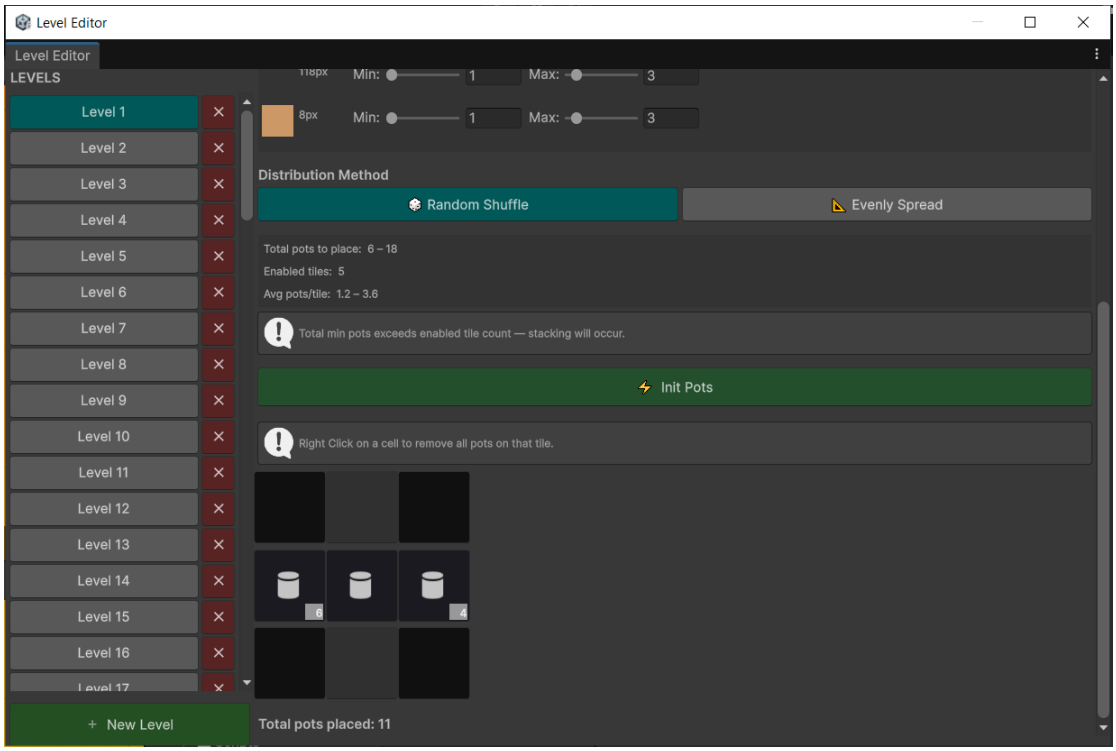


Pot Mode

Places colored pots onto enabled grid tiles. Requires image data to be initialized first.

- **Per-Color Config** — For each palette color, Min Pots and Max Pots sliders (1–20) define the range of pots that can be placed for that color.
- **Distribution Method** — Two options: Random Shuffle (pots placed randomly across tiles) or Evenly Spread By Color (greedy max-distance algorithm spreads same-color pots as far apart as possible).
- **Live Info** — Shows the total pot range, enabled tile count, and average pots per tile for a quick sanity check.
- **Init Pots** — Runs the selected distribution algorithm and places pots. Prompts before overwriting existing pots.
- **Grid interaction** — Right-click a cell to remove all pots on that tile. A count badge appears on cells with multiple pots.

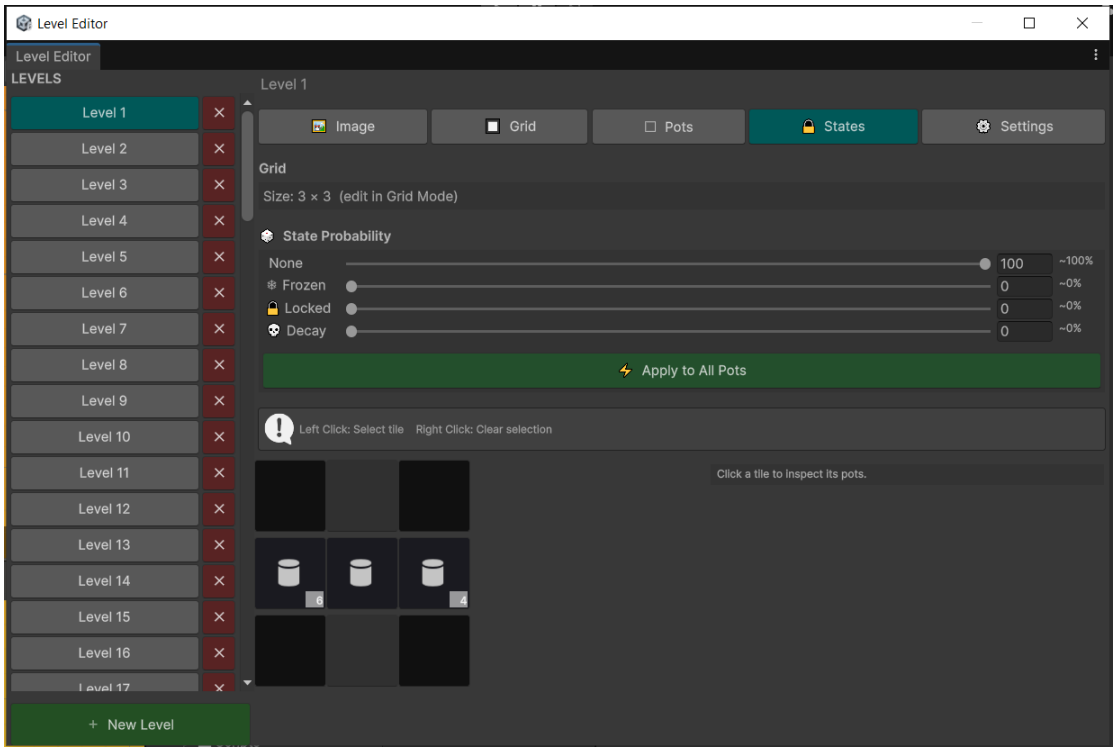




State Mode

Assigns special states to individual pots. Requires pots to be initialized first.

- **State Probability panel** — Weighted sliders for None, Frozen ❄, Locked 🔒, and Decay 💀 states. When Decay weight > 0, Min/Max Decay Time fields appear. Apply to All Pots randomizes all pot states according to these weights.
- **Grid interaction** — Left-click a tile to select it; right-click to clear selection. The selected tile's pots appear in a side panel.
- **Side panel** — For each pot on the selected tile: a color swatch, pot ID, and a state dropdown. When Decay is chosen, a decay time slider (1–60 s) and preset buttons (5s / 10s / 30s / 60s) appear.

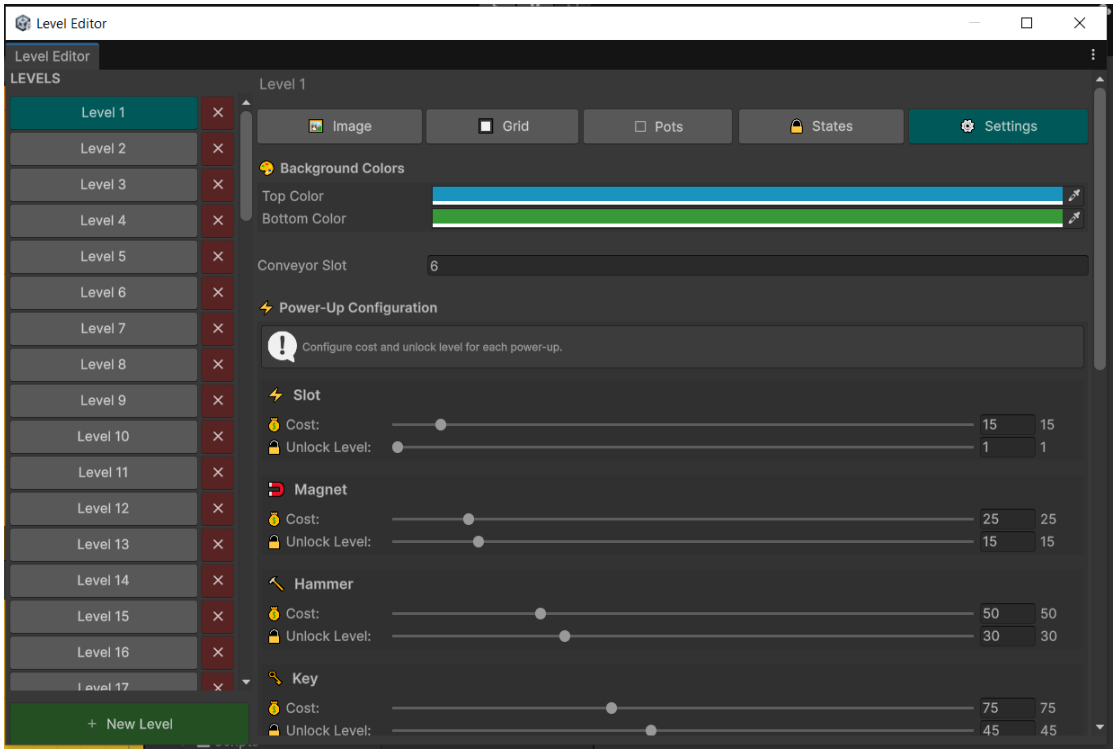


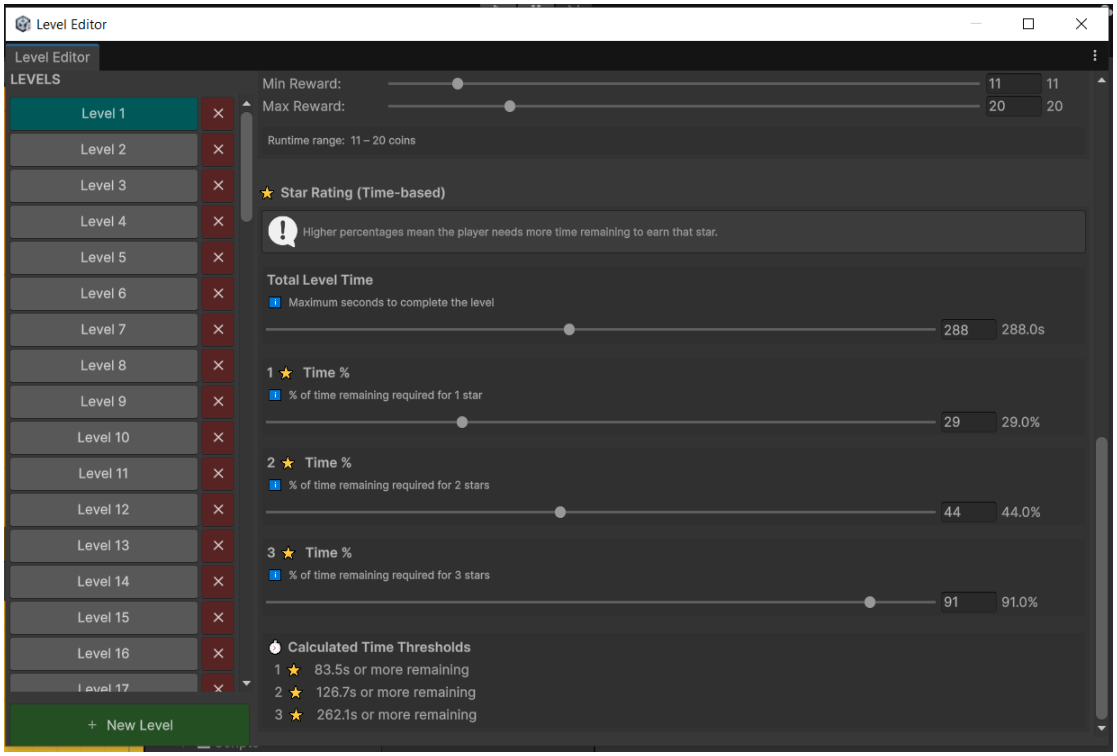
Settings Mode

Configures level-wide game parameters. This mode does not use the interactive grid.

- **Background Colors** — Top and bottom gradient colors for the level background.
- **Conveyor Slot** — Integer field controlling the number of conveyor slots available in the level.
- **Power-Up Configuration** — Per power-up cost (0–200) and unlock level (1–100) sliders. Supported types: Slot ⚡, Magnet □, Hammer 🔨, Key 🔑, Cooler ❄. An Initialize Default Power-Ups button appears if the list is empty.
- **Coin Reward** — Min and max coins awarded per pot cleared. A random value in this range is chosen at runtime.

- **Star Rating** — Total level time (30–600 s) and percentage-of-time-remaining thresholds for 1, 2, and 3 stars. Calculated second thresholds are displayed live.

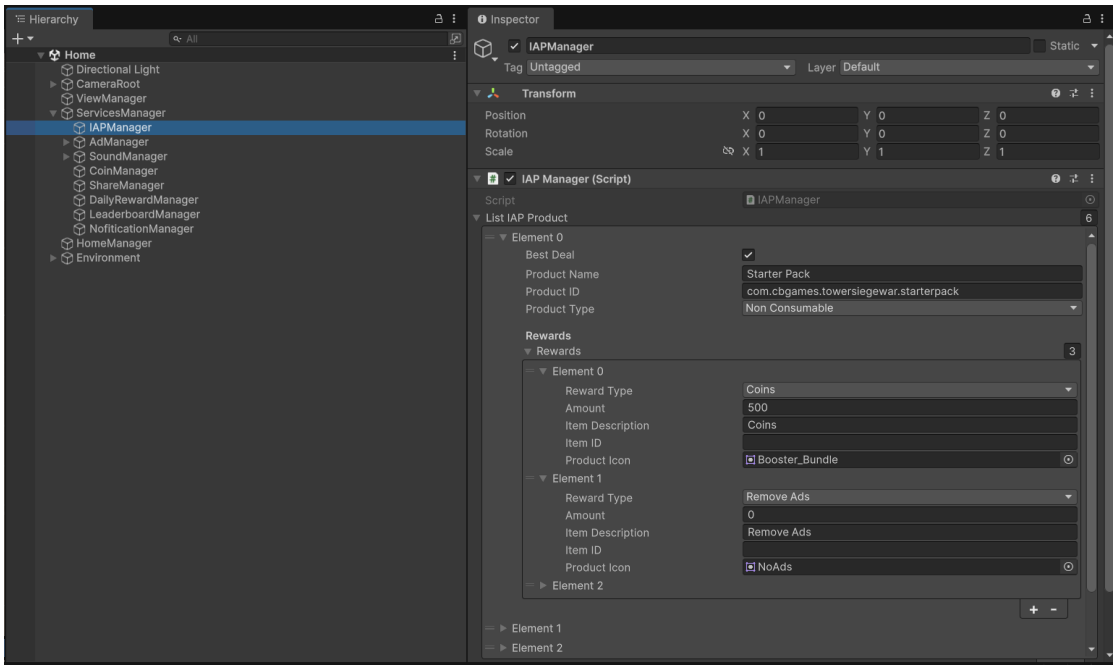




All changes save automatically via EditorUtility.SetDirty + AssetDatabase.SaveAssets and are fully undoable with Ctrl/Cmd-Z.

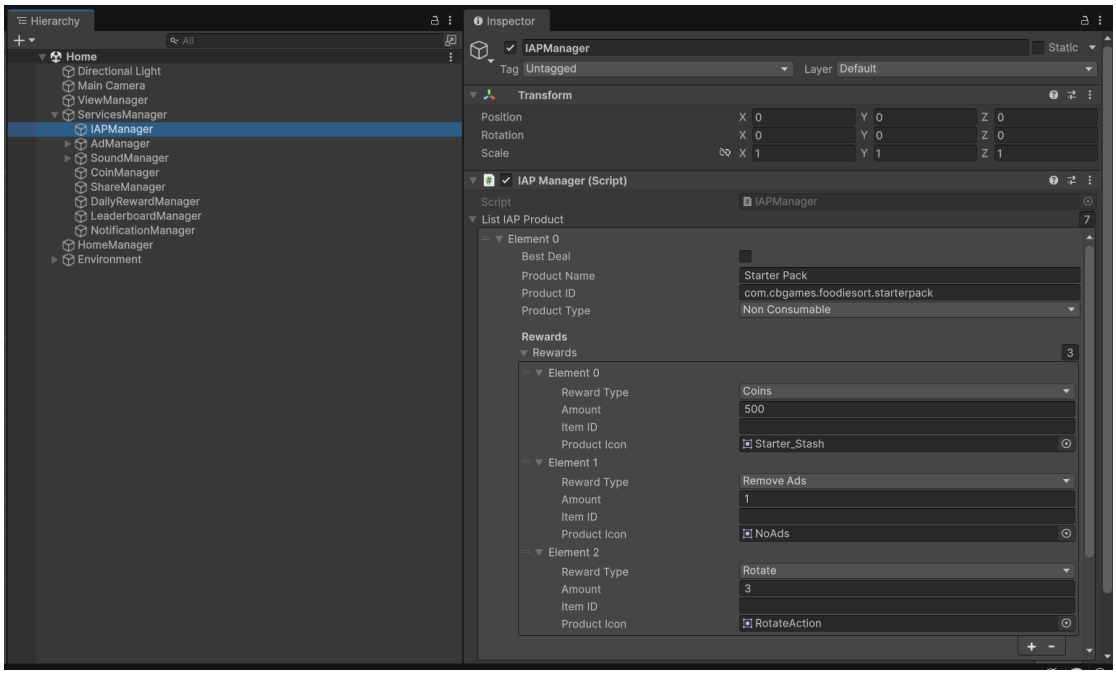
VI. SERVICES CONFIGURATION

This template comes with many features like multiple ads, leaderboard, daily reward, notification…ect. You can find all the scripts for these features under ServicesManager object in Home scene.



1. IAP Manager

The IAPManager object in hierarchy of scene Home contains IAPManager component, in which you can customize the iap products you want to use. Main container for in-app purchase product configuration.



- ❖ *bestDeal (bool)*

- Marks this product as the "best deal" for promotional display
- Shows special badge/highlight in the shop UI
- Default: false

❖ *productName (string)*
- Display name shown to users in the store
- Example: "Mega Coin Pack", "Premium Bundle"

❖ *productID (string)*
- Unique identifier matching your store configuration (App Store/Google Play)
- Example:"com.yourgame.coins_1000", "com.yourgame.remove_ads"
- Must match exactly with store console settings

❖ *productType (ProductType enum)*
- Type of purchase behavior
- Consumable: Can be purchased multiple times (coins, gems)
- NonConsumable: One-time purchase (remove ads, permanent unlocks)
- Subscription: Recurring purchase (monthly pass)

❖ *rewards (List<ProductReward>)*
- Collection of items/bonuses the player receives upon purchase
- Can contain multiple rewards per product
- See ProductReward section below

ProductReward Class

Defines individual rewards included in an IAP product.

❖ *rewardType (RewardType enum):* category of reward:
- Coins: In-game currency
- RemoveAds: Ad removal feature
- Character: Playable character unlocks
- Tower: Tower/building unlock

❖ *amount (int)*
- Quantity of the reward
- Used for: Coins, gems, consumables
- Set to 0 for non-quantity items (RemoveAds, Characters)
- Example: 1000 for 1000 coins

❖ *itemDescription (string)*
- User-friendly description of the reward
- Example: "Epic Warrior Hero", "Golden Tower Defense"
- Displayed in purchase confirmation/reward screens

❖ *itemID (string)*
- Internal identifier for the specific item

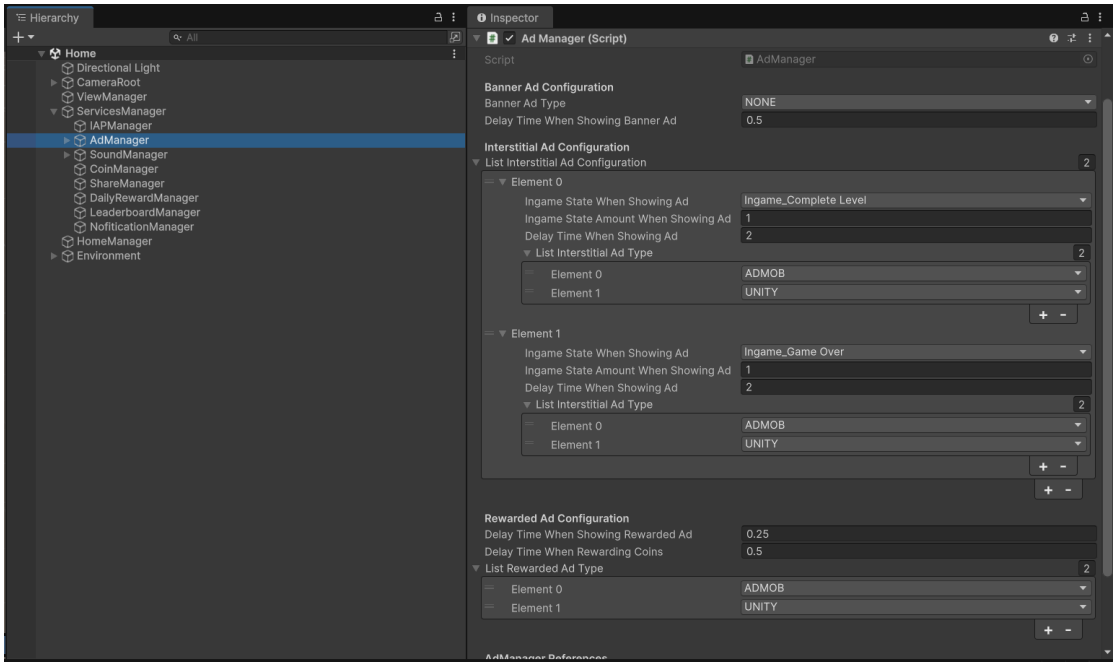
- Used for: Characters, towers, skins, unique items
- Leave empty for currency/generic rewards
- Example: "character_knight_001", "tower_cannon_gold"

❖ *productIcon (Sprite)*

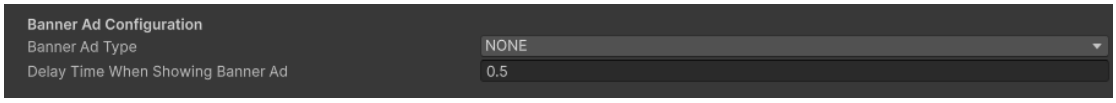
- Visual icon representing the reward
- Displayed in shop UI and reward notifications
- Assign in Unity Inspector

2. Ad Manager

The AdManager object in hierarchy of scene Home contains AdManager component, in which you can customize which type of ads you want to use and how you want to show the ads.



Banner Ad Configuration: this is the section where you can control which type of banner ad you want to show. Currently, the template support for 2 types of banner ad: Admob and Unity.

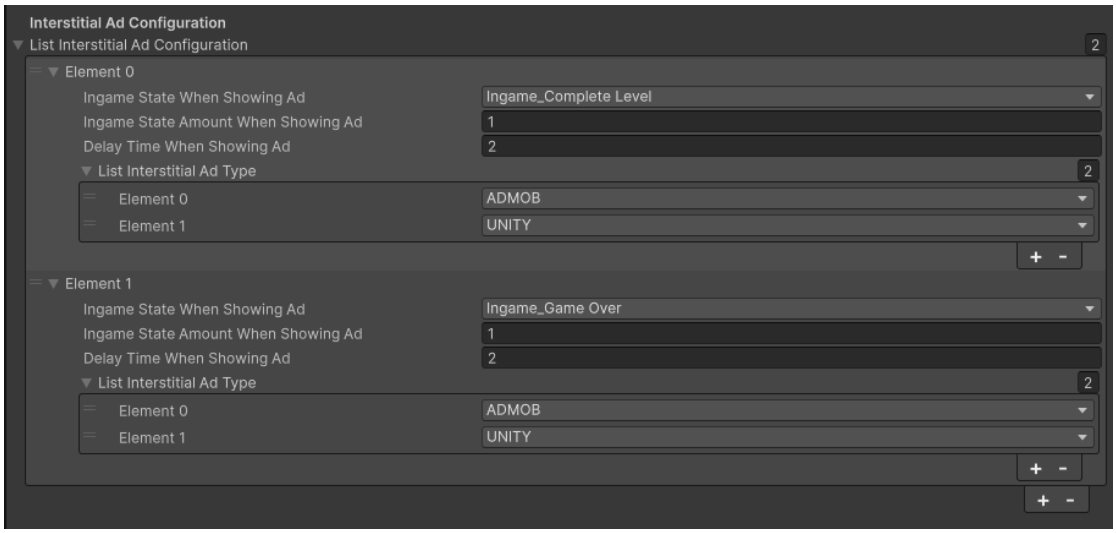


❖ *Banner Ad Type:* the type of banner ad you want to show.

❖ *Delay Time When Showing Banner Ad:* the delay time when showing banner ad.

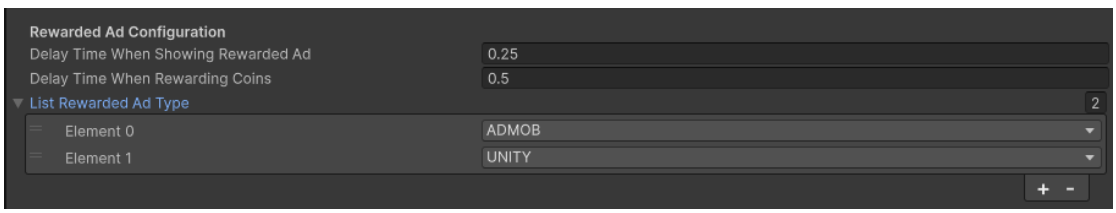
Interstitial Ad Configuration: this is the section where you can control which type of interstitial ad you want to show and how you want to show it. Currently, the template support for 2 types of interstitial ad: Admob and

Unity.



- *Ingame State When Showing Ad:* the game state you want to show the ad.
- *Ingame State Amount When Showing Ad:* the amount of game state that player go through to show ad. Example: if the value is 2 and *Ingame State When Show Ad* is Ingame_GameOver, that mean the ad will show after 2 times of game over.
- *Delay Time When Showing Ad:* the delay time when showing the interstitial ad.
- *List Interstitial Ad Type:* the list of interstitial ad type. The first item of this list will be the first priority of interstitial ad. Example: in the image you can see 2 items: ADMOB and UNITY. ADMOB is the first item so, *Ad Manager* will load the interstitial ad of Admob first, if there's no Admob's interstitial ad to load, then *Ad Manger* will continue to load Unity interstitial ad.

Rewarded Ad Configuration: this is the section where you can control which type of rewarded ad you want to show and how you want to show it. Currently, the template support for 2 types of rewarded ad: Admob and Unity.

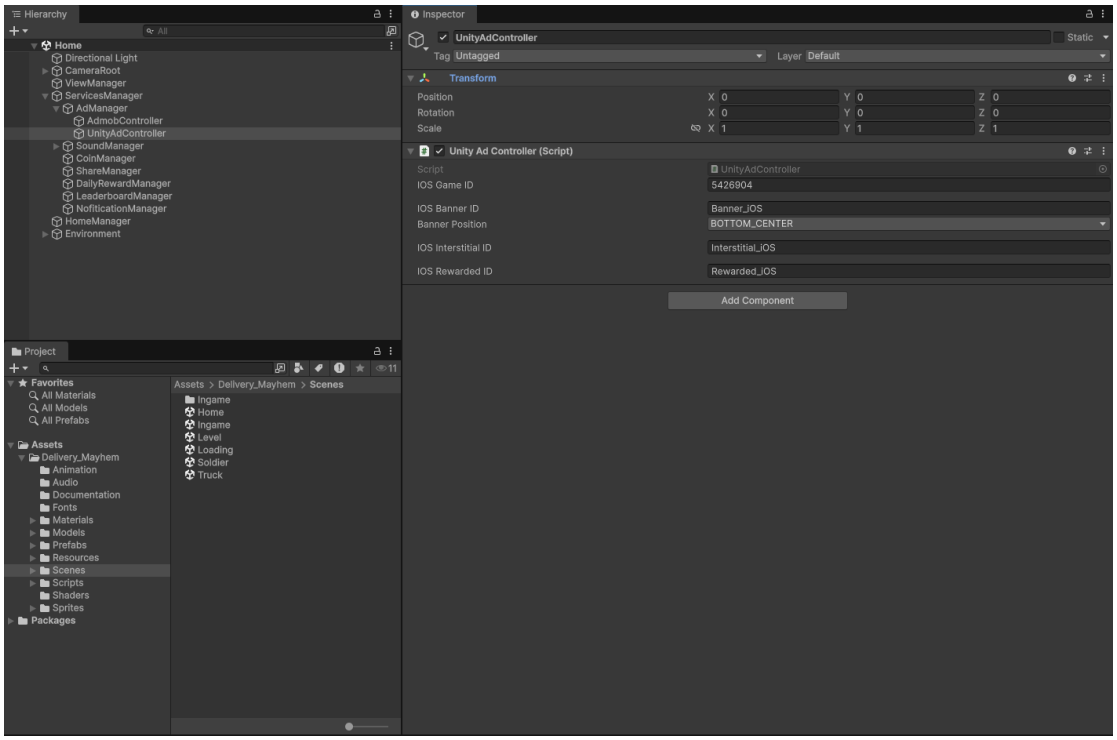


- *Delay Time When Showing Rewarded Ad:* the delay time when showing rewarded ad.
- *Delay Time When Rewarding Coins:* the delay time when rewarding coins to user after they watched rewarded ad.
- *List Rewarded Ad Type:* the list of rewarded ad type. The first item of this list will be the first priority of rewarded ad. Example: in the

image you can see 2 items: ADMOB and UNITY. ADMOB is the first item so, *Ad Manager* will load the rewarded ad of Admob first, if there's no Admob's rewarded ad to load, then *Ad Manger* will continue to load Unity rewarded ad.

3. Unity Ad Controller

Before setting up Unity ad, you need to set up your project for Unity Services first, you can see the instruction here. After setting up Unity Services, just put your game id and your ad ids and you will good to go.

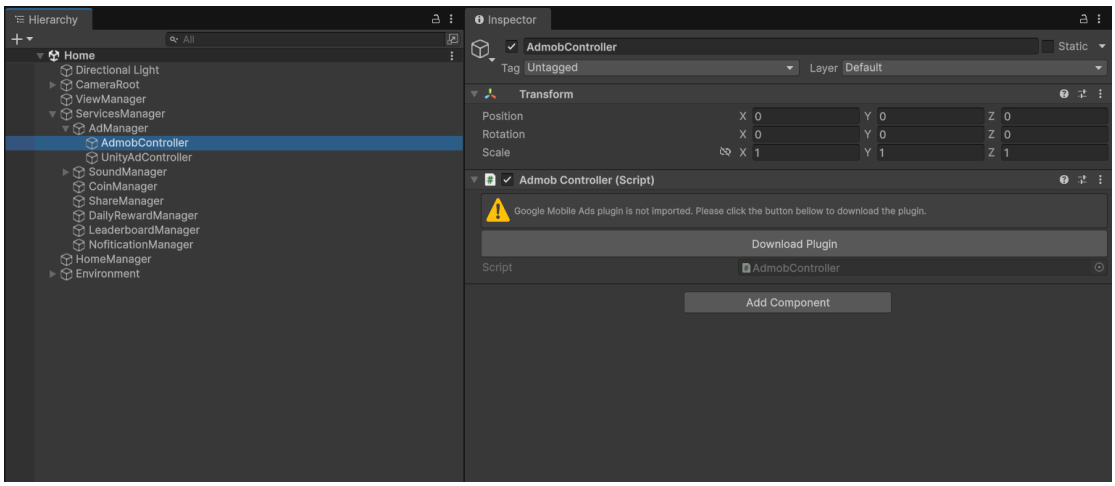


- *Android Game ID:* the android id of your unity ad project (it will show the id for iOS when you switch to iOS). You can find all of your ad projects here.
- *Android Banner ID:* the android banner id of your ad project (it will show the id for iOS when you switch to iOS).
- *Banner Position:* the position of the banner ad.
- *Android Interstitial ID:* the interstitial id of your ad project (it will show the id for iOS when you switch to iOS).
- *Android Rewarded ID:* the rewarded ad id of your ad project (it will show the id for iOS when you switch to iOS).

4. Admob Controller

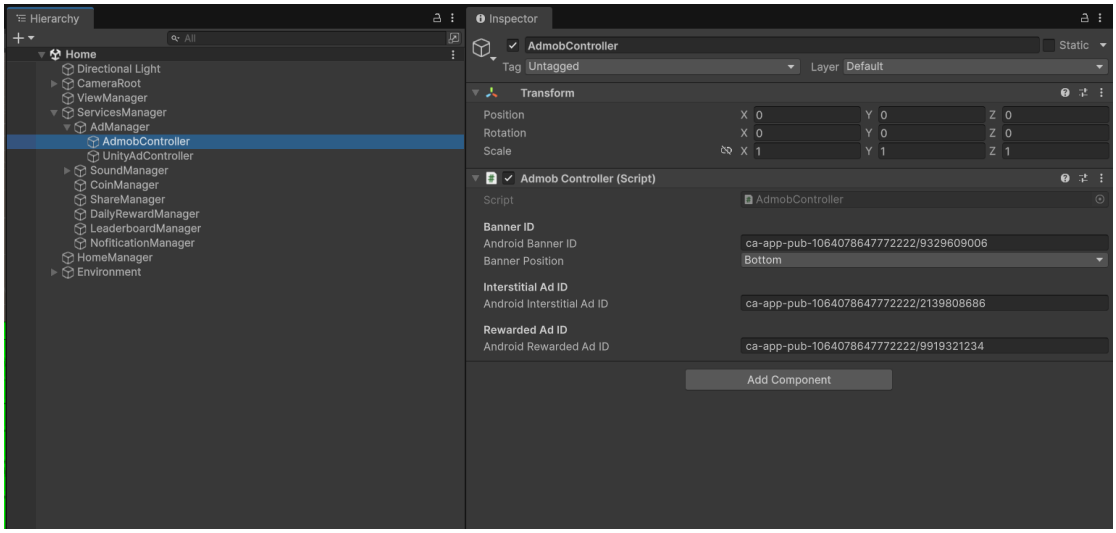
The AdmobManager object in hierarchy of scene Home contains

AdmobController component, in which you can customize parameters like admob id, ad units...



As you can see in the image, there's no option to change ads units because the template did not have google mobile ads plugin included, so if you want to use Admob, please click to the Download Plugin button, download the latest version of google mobile ads and import it into the project.

After imported the plugin, select AdManager object in the hierarchy, wait for few seconds for the project rebuild and you will see this.



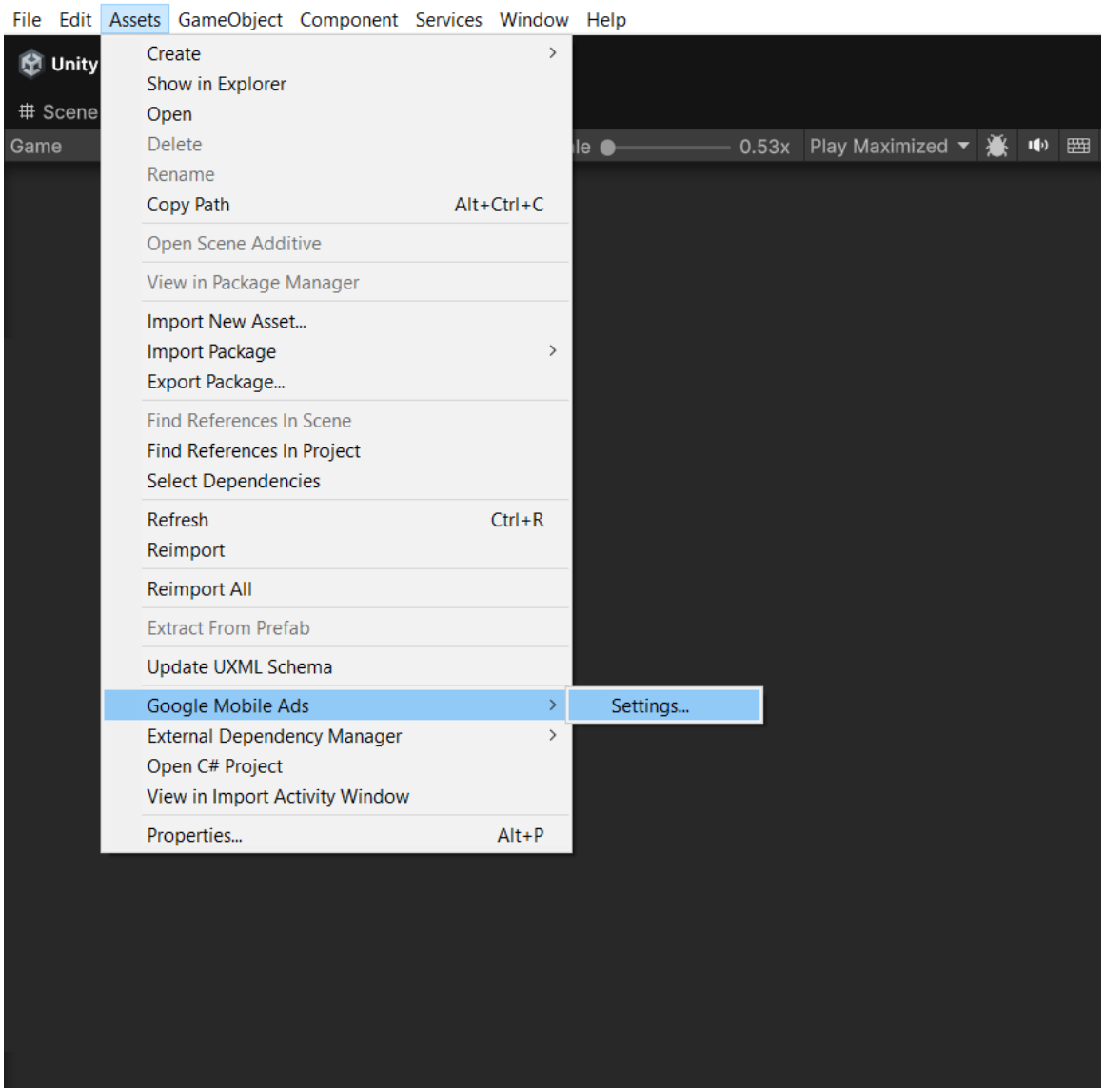
Now you can config Admob id and all the ad units as you want. Currently, the platform using is Android, that why all the ids in the image showing for Android, of course it will show the ids for iOS when you switch the build platform to iOS.

- *Android Banner ID:* the id of banner ad in your Admob account (it will show the id for iOS when you switch to iOS).
- *Banner Position:* the position of the banner ad.
- *Android Interstitial ID:* the id of interstitial ad in your Admob account (it will show the id for iOS when you switch to iOS).
- *Android Rewarded ID:* the id of rewarded ad in your Admob

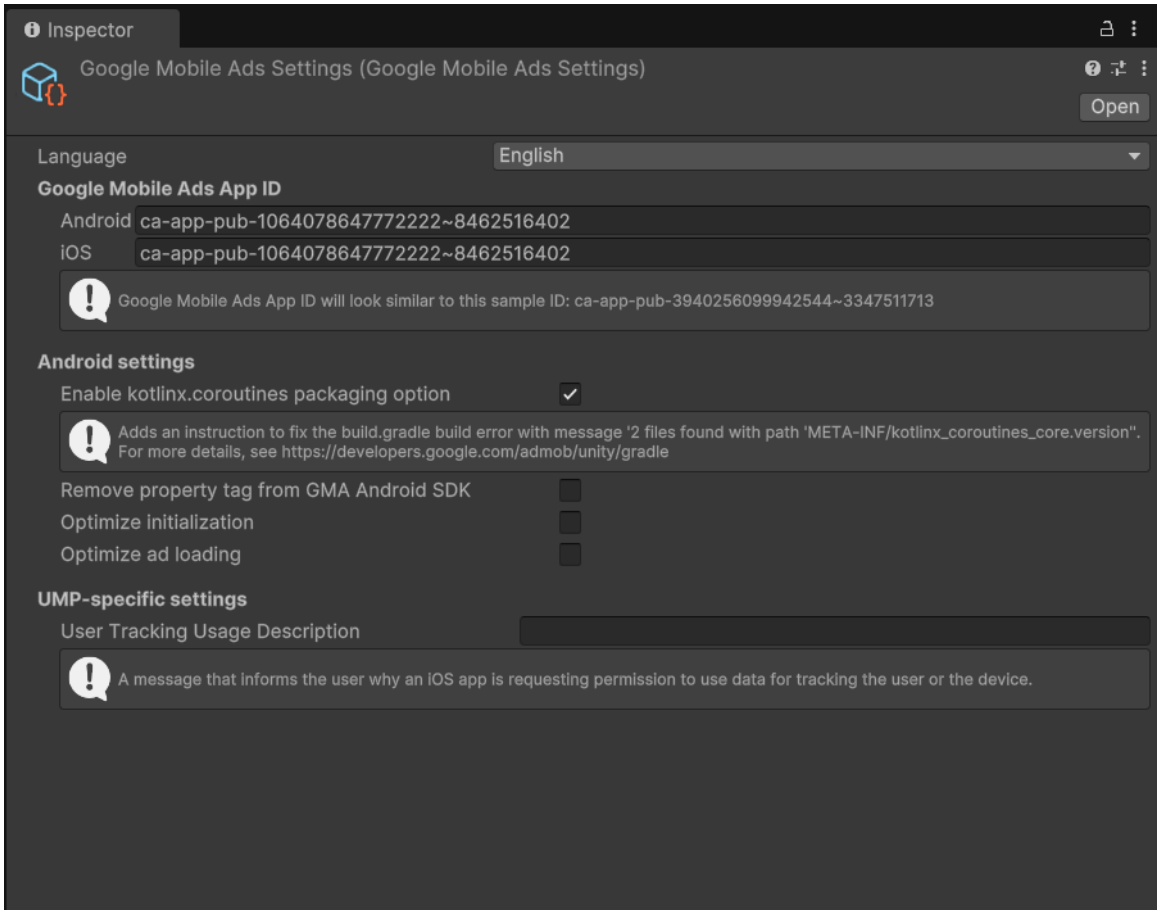
account (it will show the id for iOS when you switch to iOS).

Now we need to setup the Admob App ID. Please follow these steps to setup the Abmob App ID.

- Go to Assets/Google Mobile Ads/Setting



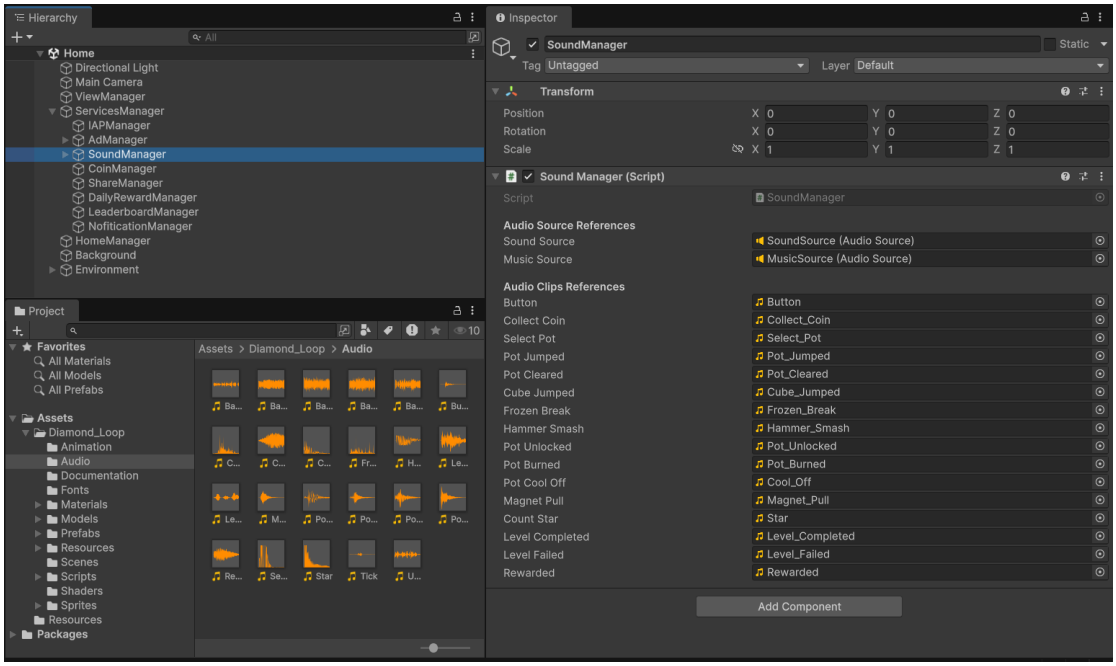
- Insert the Google Mobile Ads App ID at Android/iOS field base on your current platform.



➢ Hit Ctrl + S to save, that's all.

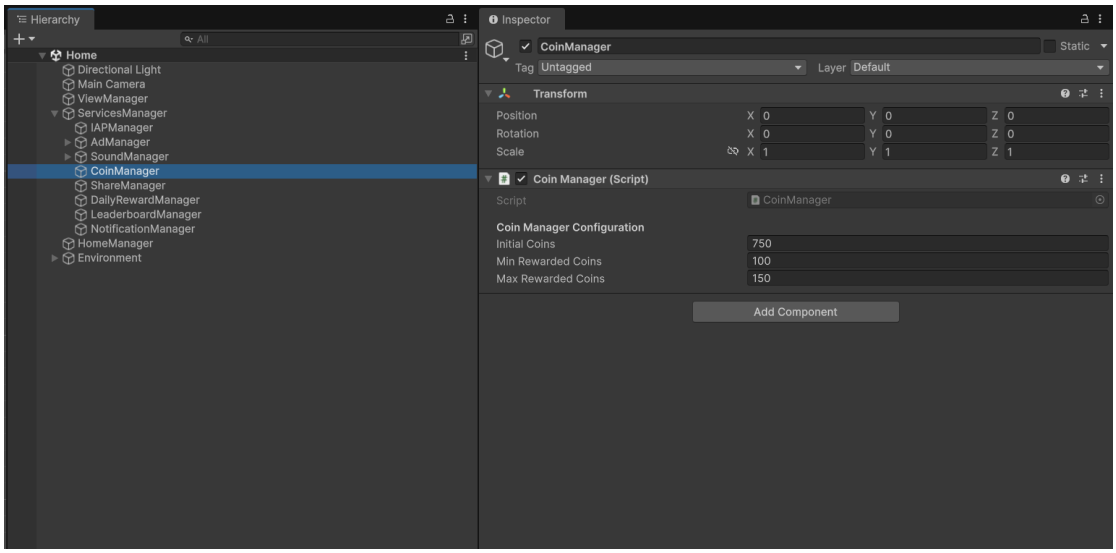
5. Sound Manager

All sounds included in this game are free-to-use in commercial projects and are located under the path *Assets/Diamond_Loop/Audio* folder.



6. Coin Manager

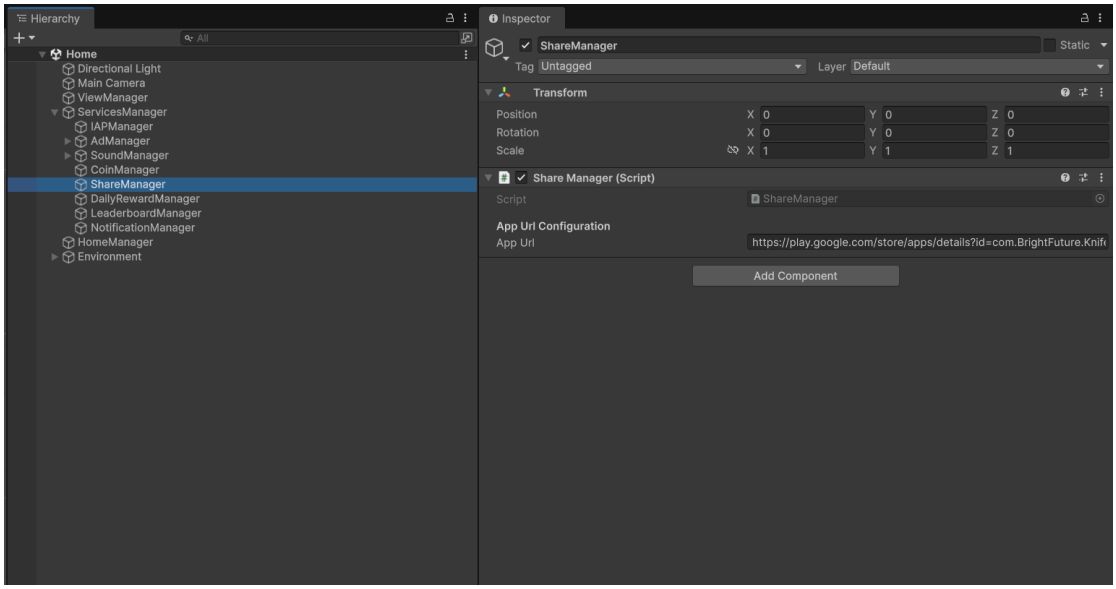
Coin is the currency in the game that player collect to unlock new characters. All information about coin can be config in CoinManager game object.



- *Initial Coins:* the initial coins that player will have when the game start at the first time.
- *Min Rewarded Coins:* the minimum coin amount that user will be rewarded after they watched rewarded ad.
- *Max Rewarded Coins:* the maximum coin amount that user will be rewarded after they watched rewarded ad.

7. Share Manager

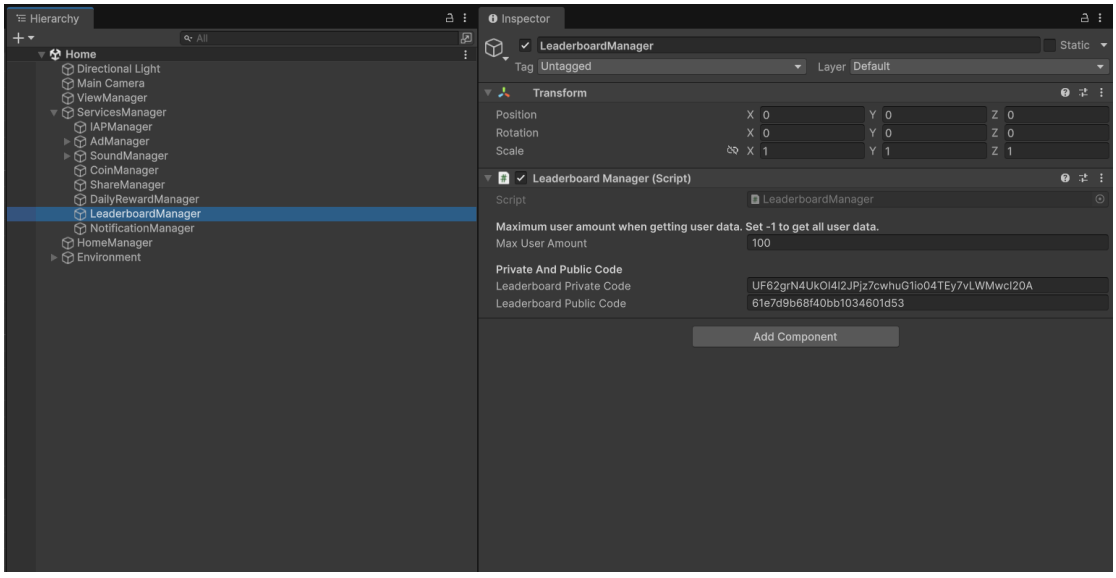
All information for sharing feature can be config in ShareManager game object. It contains the url of your app. You can config these features from the ShareManager object in the hierarchy.



- *App Url:* the url of the app (Google Play on Android and App Store on iOS).

8. Leaderboard Manager

All information for leaderboard feature can be config in LeaderboardManager game object. It contains 2 parameters for you to config the leaderboard. Currently, we used Dreamlo to handle leaderboard feature.

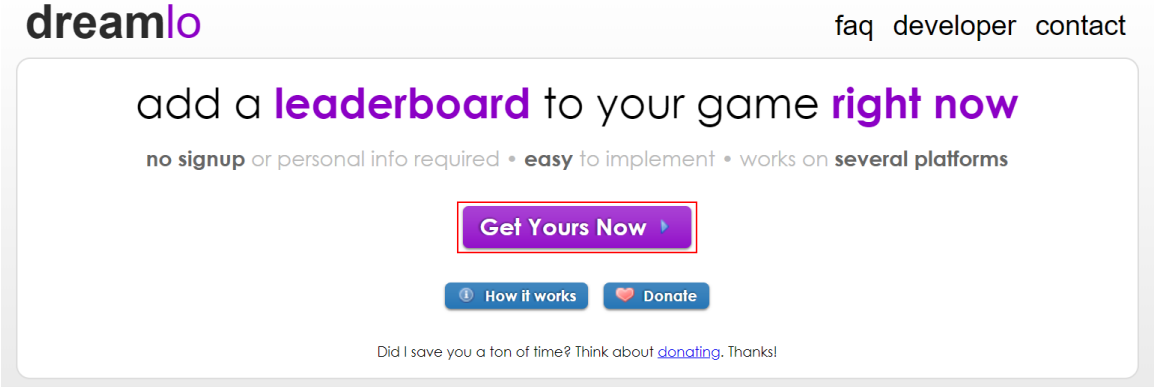


- *Max User Amount:* the maximum amount user show in the leaderboard. If you set it to -1, it will show all the user that played the game and report to leaderboard table. We suggest you to limit it to 50 or 100, because if your game have 1000 downloads, which mean you have 1000 user in the leaderboard and when you load it

all to the leaderboard view, the game will become laggy.

- *Leaderboard Private Code:* the leaderboard private code of Dreamlo.
- *Leaderboard Public Code:* the leaderboard public code of Dreamlo.

Now we will show you how to get the private and public code from Dreamlo. First, you need to go to Dreamlo.com, at the top of the front page, click to Get Yours Now button.



After that, you will be direct to a page that you can get your private and public code.

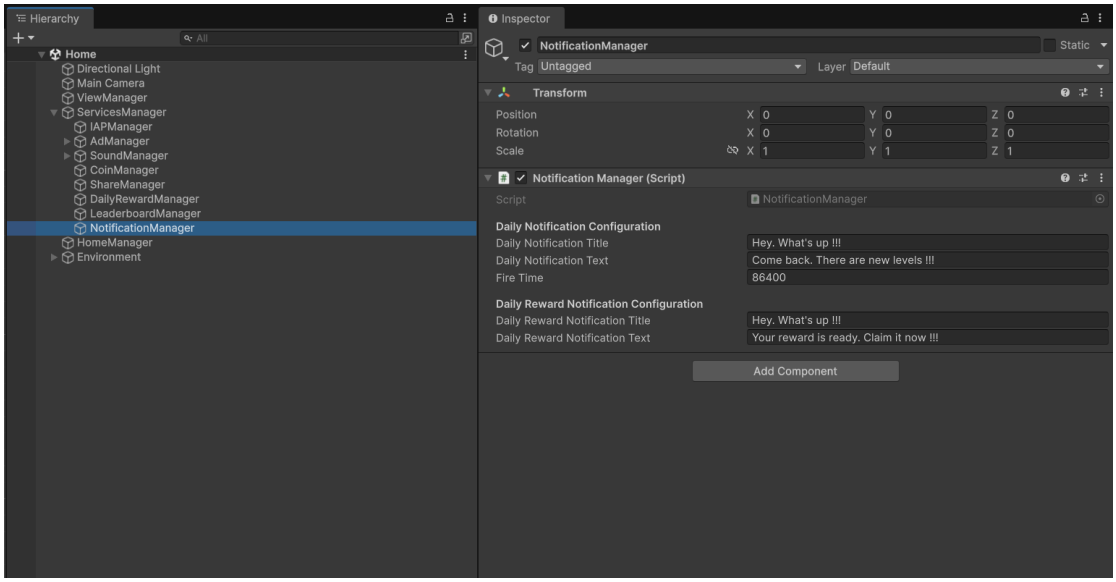


Copy 2 codes and paste it on LeaderboardManager component. That's it.

9. Notification Manager

All information for notification feature can be config in

NotificationManager game object. It contains 3 parameters for you to config the notification. Currently, we used Mobile Notification package provided by Unity to handle the notification.



Daily Notification Configs: this is the notification that fire daily.

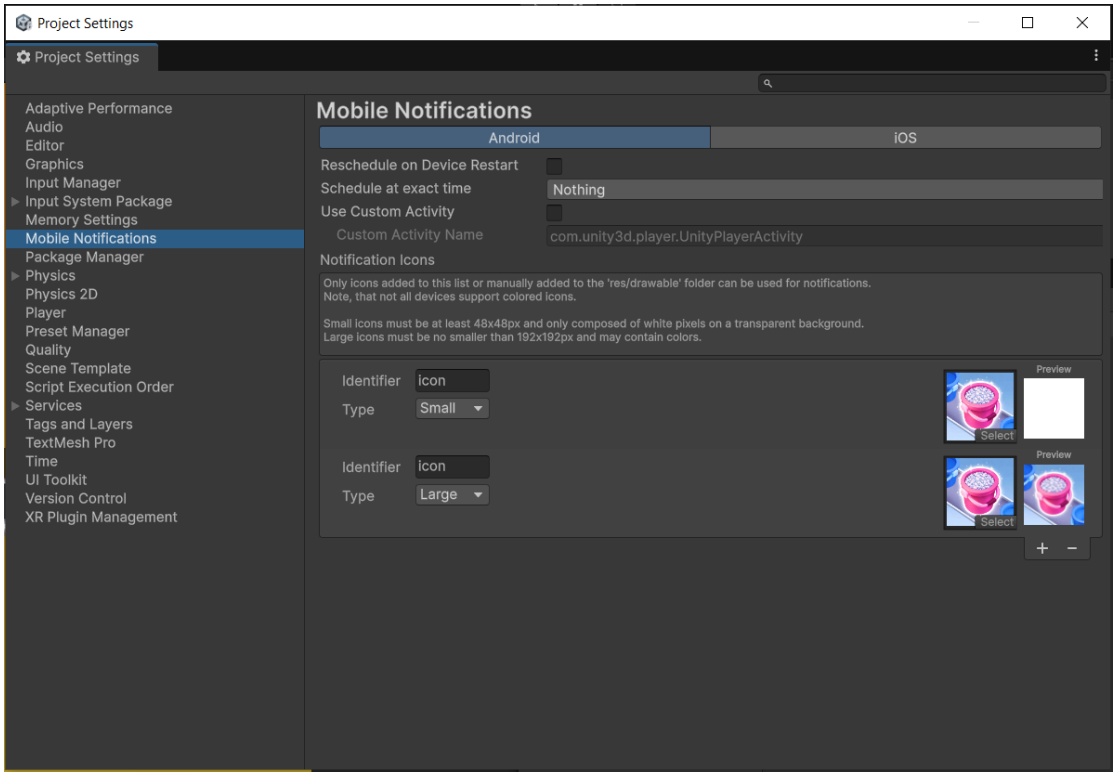
- *Daily Notification Tile:* the tile of the notification when it fired.
- *Daily Notification Text:* the text of the notification when it fired.
- *Fire Time:* how long till the notification get fire (in seconds) after the game close. Currently it's 86400 seconds which is 1 day.

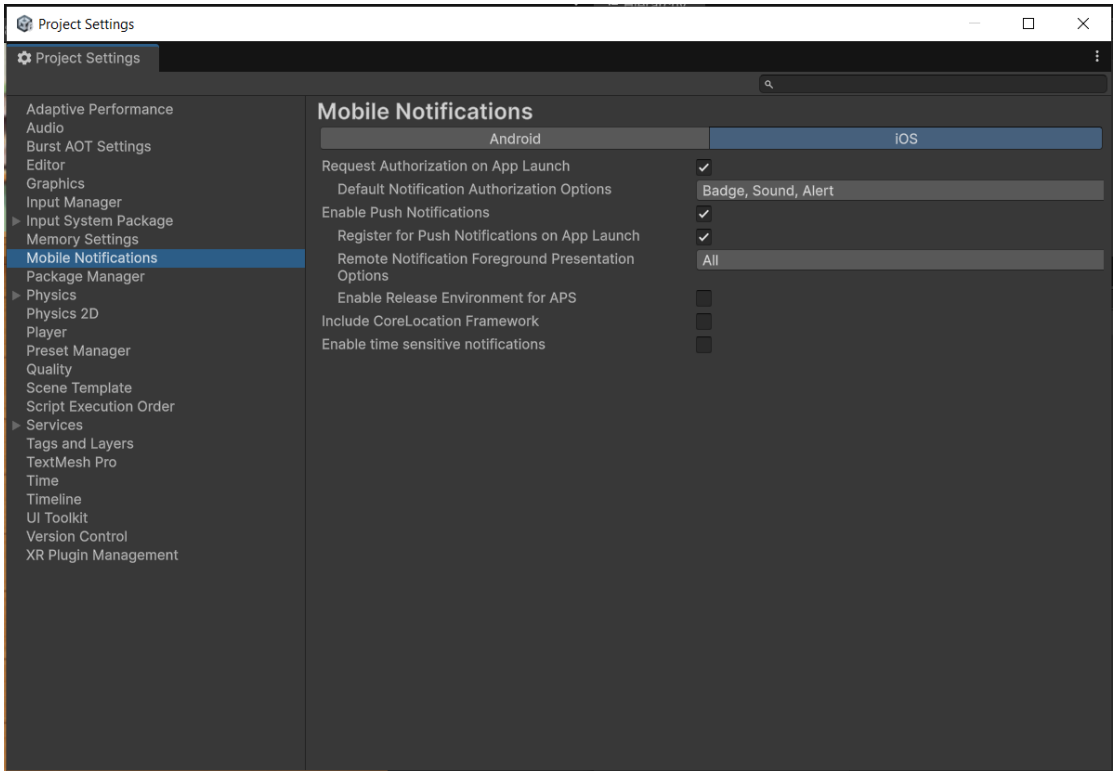
Reward Coins Notification Configs: this is the notification that fire when a reward is ready to claim.

- *Reward Coins Notification Tile:* the tile of the notification when it fired.
- *Reward Coins Notification Text:* the text of the notification when it fired.

You can set up the notification icon by following these steps.

Go to File -> Project Setting -> Mobile Notification

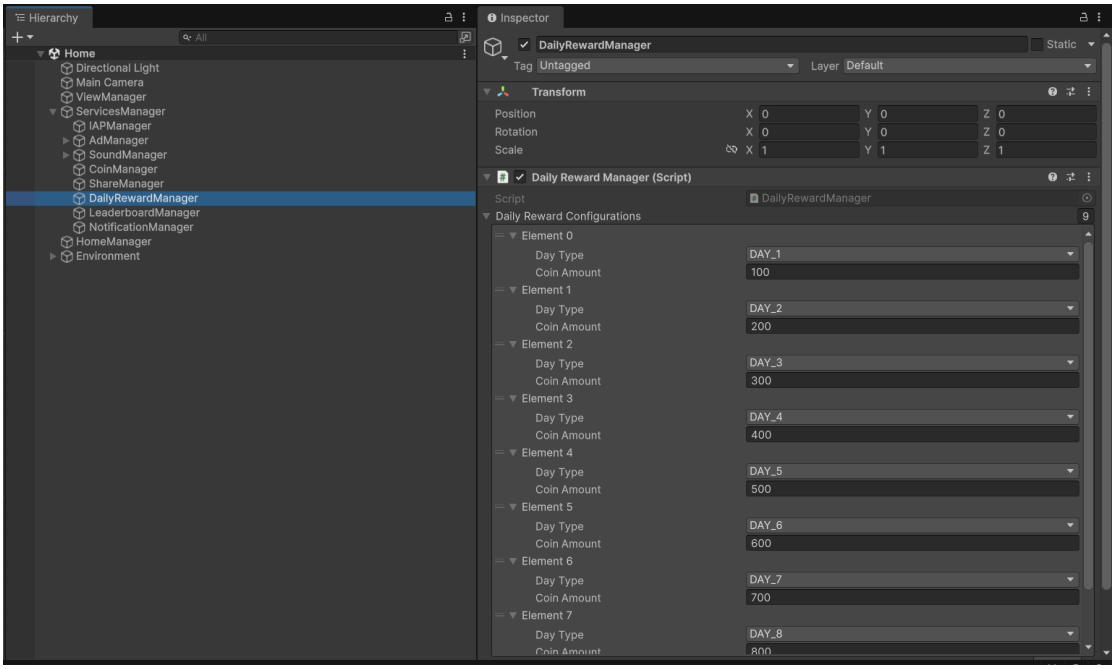




As you can see, there's a filed for you to setup icon for notification. Remember, you must set the Identifier as "icon".

10. Daily Reward Manager

This template has a built-in daily reward system in which the user will be rewarded with coins every predefined interval of time. This is an effective way to increase user engagement and retention for your game. You can configure this feature from the *DailyRewardManager* object in the hierarchy.



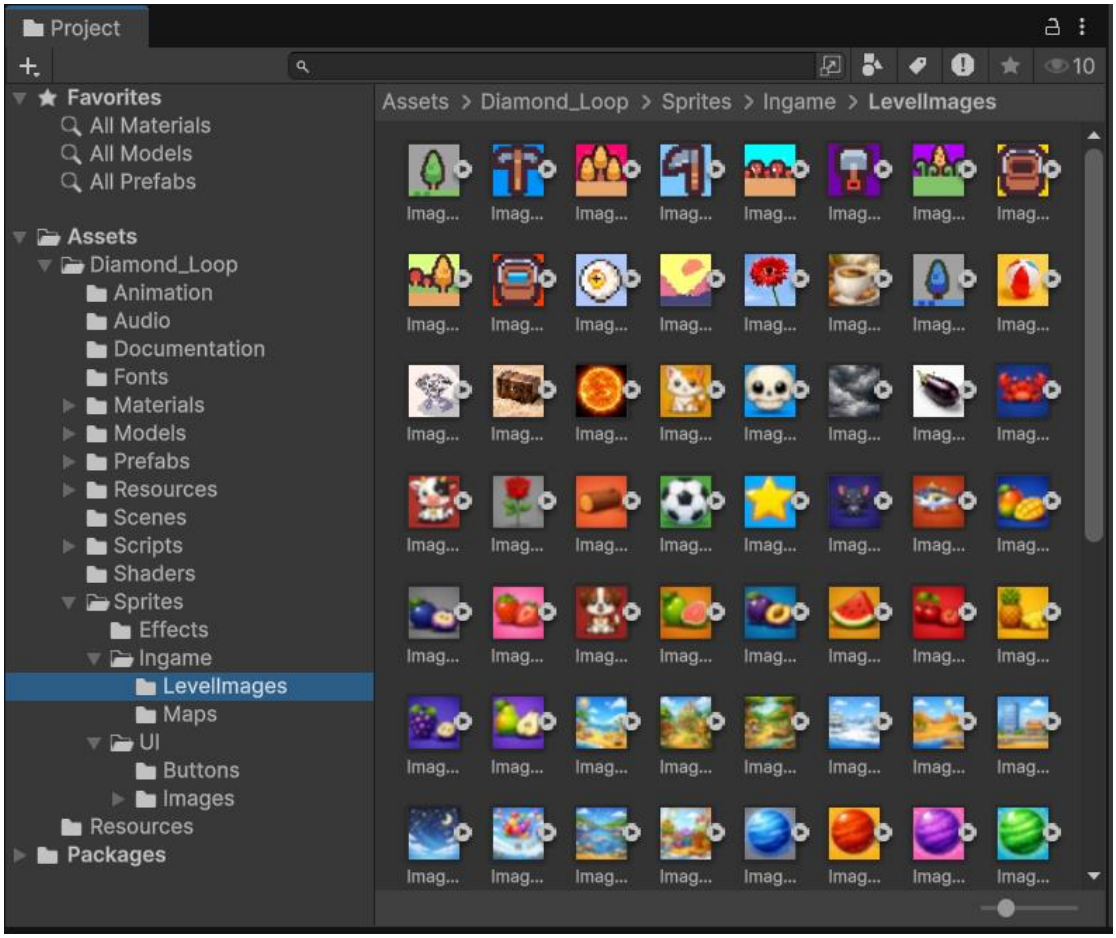
***Daily Reward Configurations**:* this array allows you config the rewarded coin amount for each day.

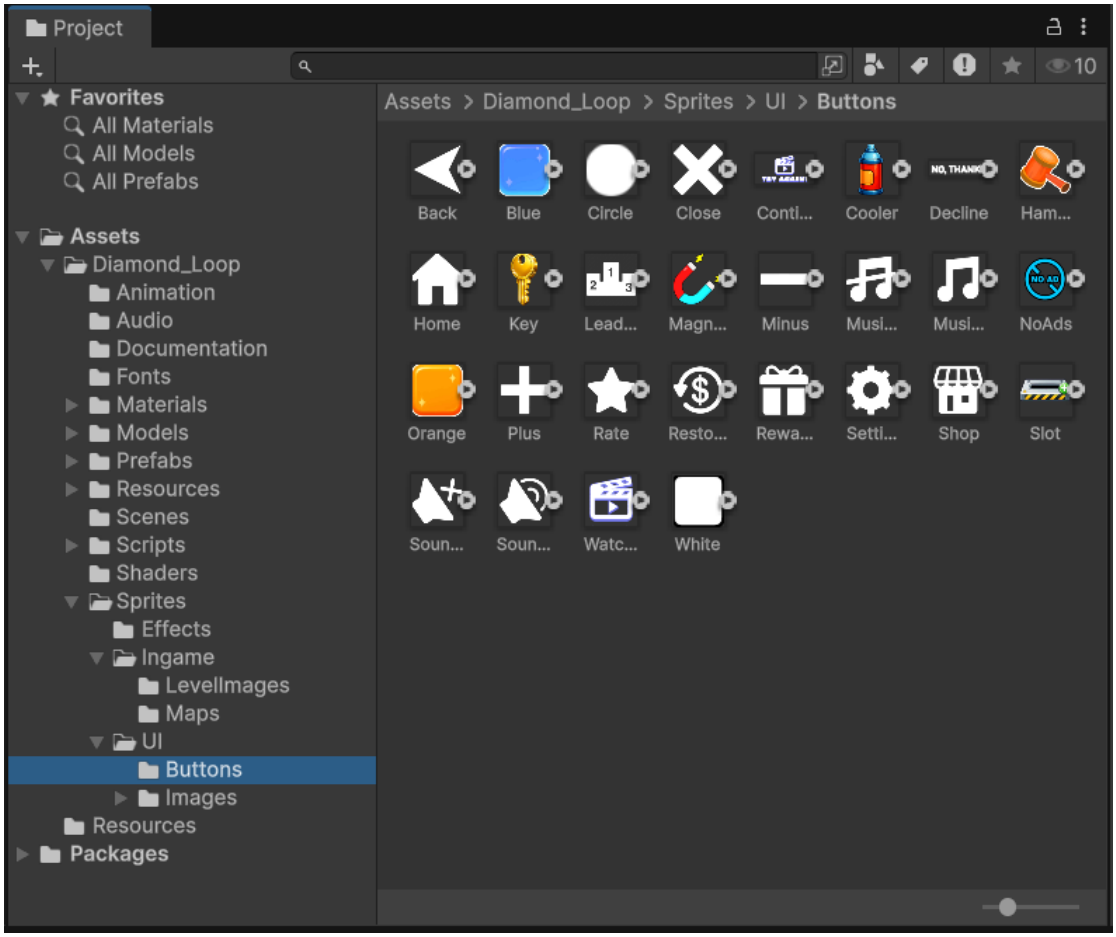
- *Day Type:* the day of the reward.
- *Coin Amount:* the amount of coin player will be rewarded.

***IMPORTANT:** Currently, the daily reward system was designed for only 9 days, after player get all the reward (claimed 9 rewards), the daily reward system will reset to day 1. DO NOT ADD MORE ELEMENT TO THE ARRAY, IT MAY CAUSE UNEXPECTED ERRORS.

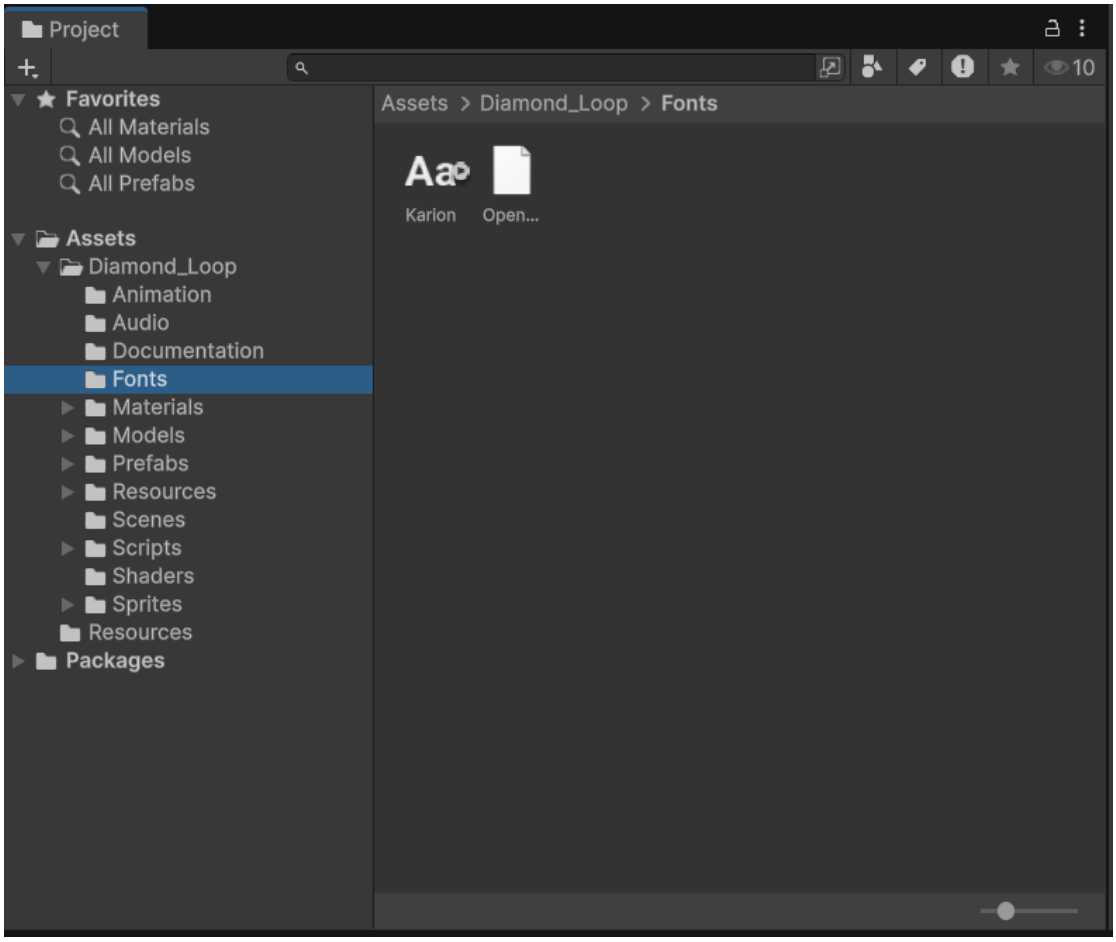
11. Customizing UI

All sprites used in this game (for buttons and other UI components) are located under the path *Assets/Diamond_Loop/Sprites* folder. You can replace them with your own sprites to modify the images as you like.


Project
Favorites
All Materials
All Models
All Prefabs
Assets
Diamond_Loop
Animation
Audio
Documentation
Fonts
Materials
Models
Prefabs
Resources
Scenes
Scripts
Shaders
Sprites
Effects
Ingame
LevelImages
Maps
UI
Buttons
Images
Resources
Packages
Assets > Diamond_Loop > Sprites > Ingame > LevelImages



All fonts used in this game are free-to-use in commercial projects. Fonts are located under the path *Assets/Diamond_Loop/Fonts* folder together with appropriate license files.


Project
Favorites
All Materials
All Models
All Prefabs
Assets
Diamond_Loop
Animation
Audio
Documentation
Fonts
Materials
Models
Prefabs
Resources
Scenes
Scripts
Shaders
Sprites
Resources
Packages
Assets > Diamond_Loop > Fonts
Karion
Open...